第15章　序列生成模型

> 人类语言似乎是一种独特的现象，在动物世界中没有显著类似的存在.
>
> ——诺姆·乔姆斯基（Noam Chomsky）
> 美国语言学家、哲学家

在深度学习的应用中，有很多数据是以序列的形式存在，比如声音、语言、视频、DNA序列或者其他的时序数据等. 以自然语言为例，一个句子可以看作符合一定自然语言规则的词（word）的序列. 在认知心理学上有一个经典的实验，让一个人看下面两个句子：

这里假定语言的最基本单位为词（word），当然也可以为字或字母（character）.

面包上涂黄油，
面包上涂袜子.

后一个句子在人脑进行语义整合时需要更多的处理时间，说明后一个句子更不符合自然语言规则. 这些语言规则包含非常复杂的语法和语义的组合关系，我们很难显式地建模这些规则. 为了有效地描述自然语言规则，我们可以从统计的角度来建模. 将一个长度为 T 的文本序列看作一个随机事件 $X_{1:T} = \langle X_1, \cdots, X_T \rangle$，其中每个位置上的变量 X_t 的样本空间为一个给定的词表（vocabulary）$\mathcal{V}$，整个序列 $\boldsymbol{x}_{1:T}$ 的样本空间为 $|\mathcal{V}|^T$. 在某种程度上，自然语言也确实有很多随机因素. 比如当我们称赞一个人漂亮时，可以说“美丽”“帅”或者“好看”等. 当不指定使用场合时，这几个词可以交替使用，具体使用哪个词相当于一个随机事件. 一个文本序列的概率大小可以用来评估它符合自然语言规则的程度.

给定一个序列样本 $\boldsymbol{x}_{1:T} = x_1, x_2, \cdots, x_T$，其概率是 T 个词的联合概率：

在本章中，我们用 X_t 表示位置 t 上的随机变量，$\boldsymbol{x}_{1:T}$ 表示一个序列样本，x_t 表示一个序列样本在位置 t 上的值.

$$p(\boldsymbol{x}_{1:T}) \triangleq P(\boldsymbol{X}_{1:T} = \boldsymbol{x}_{1:T}) \tag{15.1}$$

$$= P(X_1 = x_1, X_2 = x_2, \cdots, X_T = x_T). \tag{15.2}$$

和一般的概率模型类似，序列概率模型有两个基本问题. 1）概率密度估计：给定一组序列数据，估计这些数据背后的概率分布；2）样本生成：从已知的序列分布中生成新的序列样本.

序列数据一般可以通过概率图模型来建模序列中不同变量之间的依赖关系. 本章主要介绍在序列数据上经常使用的一种模型：自回归生成模型（AutoRegressive Generative Model）.

不失一般性，本章以自然语言为例来介绍序列概率模型.

15.1 序列概率模型

序列数据有两个特点：1）样本是变长的；2）样本空间非常大. 对于一个长度为 T 的序列，其样本空间为 $|\mathcal{V}|^T$. 因此，我们很难用已知的概率模型来直接建模整个序列的概率.

根据概率的乘法公式，序列 $\boldsymbol{x}_{1:T}$ 的概率可以写为

$$p(\boldsymbol{x}_{1:T}) = p(x_1)p(x_2|x_1)p(x_3|\boldsymbol{x}_{1:2})\cdots p(x_t|\boldsymbol{x}_{1:(t-1)}) \tag{15.3}$$

$$= \prod_{t=1}^{T} p(x_t|\boldsymbol{x}_{1:(t-1)}), \tag{15.4}$$

其中 $x_t \in \mathcal{V}, t \in \{1,\cdots,T\}$ 为词表 $\mathcal{V}$ 中的一个词，$p(x_1|x_0) = p(x_1)$.

因此，序列数据的概率密度估计问题可以转换为单变量的条件概率估计问题，即给定 $\boldsymbol{x}_{1:(t-1)}$ 时 x_t 的条件概率 $p(x_t|\boldsymbol{x}_{1:(t-1)})$.

给定一个包含 N 个序列数据的数据集 $\mathcal{D} = \{\boldsymbol{x}_{1:T_n}^{(n)}\}_{n=1}^{N}$，序列概率模型需要学习一个模型 $p_\theta(x|\boldsymbol{x}_{1:(t-1)})$ 来最大化整个数据集的对数似然函数，即

$$\max_\theta \sum_{n=1}^{N} \log p_\theta(\boldsymbol{x}_{1:T_n}^{(n)}) = \max_\theta \sum_{n=1}^{N}\sum_{t=1}^{T_n} \log p_\theta(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)}). \tag{15.5}$$

在这种序列模型方式中，每一步都需要将前面的输出作为当前步的输入，是一种自回归（AutoRegressive）的方式. 因此这一类模型也称为自回归生成模型（AutoRegressive Generative Model）.

自回归模型参见第6.1.2节.

由于 $X_t \in \mathcal{V}$ 为离散变量，我们可以假设条件概率 $p_\theta(x_t|\boldsymbol{x}_{1:(t-1)})$ 服从多项分布，然后通过不同的模型来估计. 本章主要介绍两种比较主流的自回归生成模型：N 元统计模型和深度序列模型.

多项分布参见第D.2.2.1节.

15.1.1 序列生成

一旦通过最大似然估计训练了模型 $p_\theta(x|\boldsymbol{x}_{1:(t-1)})$，就可以通过时间顺序来生成一个完整的序列样本. 令 $\hat{x}_t$ 为在第 t 步根据分布 $p_\theta(x|\hat{\boldsymbol{x}}_{1:(t-1)})$ 生成的词，

$$\hat{x}_t \sim p_\theta(x|\hat{\boldsymbol{x}}_{1:(t-1)}), \tag{15.6}$$

其中 $\hat{\boldsymbol{x}}_{1:(t-1)} = \hat{x}_1, \cdots, \hat{x}_{t-1}$ 为前面 $t-1$ 步中生成的前缀序列.

自回归的方式可以生成一个无限长度的序列. 为了避免这种情况，通常会设置一个特殊的符号 $\langle EOS \rangle$ 来表示序列的结束. 在训练时，每个序列样本的结尾都加上符号 $\langle EOS \rangle$. 在测试时，一旦生成了符号 $\langle EOS \rangle$，就中止生成过程.

束搜索 当使用自回归模型生成一个最可能的序列时，生成过程是一种从左到右的贪婪式搜索过程. 在每一步都生成最可能的词，

$$\hat{x}_t = \arg\max_{x \in \mathcal{V}} p_\theta(x|\hat{\boldsymbol{x}}_{1:(t-1)}), \tag{15.7}$$

其中 $\hat{\boldsymbol{x}}_{1:(t-1)} = \hat{x}_1, \cdots, \hat{x}_{t-1}$ 为前面 $t-1$ 步中生成的前缀序列.

这种贪婪式的搜索方式是次优的，生成的序列 $\hat{\boldsymbol{x}}_{1:T}$ 并不保证是全局最优的.

$$\prod_{t=1}^{T} \max_{x_t \in \mathcal{V}} p_\theta(x_t|\hat{\boldsymbol{x}}_{1:(t-1)}) \leq \max_{\boldsymbol{x}_{1:T} \in \mathcal{V}^T} \prod_{t=1}^{T} p_\theta(x|\boldsymbol{x}_{1:(t-1)}). \tag{15.8}$$

一种常用的减少搜索错误的启发式方法是束搜索（Beam Search）. 在每一步中，生成 K 个最可能的前缀序列，其中 K 为束的大小（Beam Size），是一个超参数. 图15.1给出了一个束搜索过程的示例，其中词表 $\mathcal{V} = \{A, B, C\}$，束大小为 2.

束搜索也经常称为集束搜索或柱搜索.

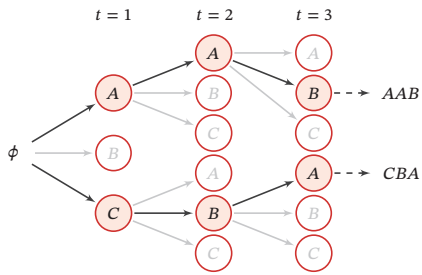


图 15.1 束搜索过程示例

束搜索的过程如下：在第 1 步时，生成 K 个最可能的词；在后面每一步中，从 $K|\mathcal{V}|$ 个候选输出中选择 K 个最可能的序列.

参见习题15-5.

束的大小 K 越大，束搜索的复杂度越高，但越有可能生成最优序列. 在实际应用中，束搜索可以通过调整束大小 K 来平衡计算复杂度和搜索质量之间的优先级.

15.2 N元统计模型

由于数据稀疏问题，当 t 比较大时，依然很难估计条件概率 $p(x_t|\boldsymbol{x}_{1:(t-1)})$. 一个简化的方法是N元模型（N-Gram Model），假设每个词 x_t 只依赖于其前面的 $N-1$ 个词（N 阶马尔可夫性质），即

马尔可夫性质参见第D.3.1节.

$$p(x_t|\boldsymbol{x}_{1:(t-1)}) = p(x_t|\boldsymbol{x}_{(t-N+1):(t-1)}). \tag{15.9}$$

当 $N=1$ 时，称为一元（Unigram）模型；当 $N=2$ 时，称为二元（Bigram）模型，以此类推.

一元模型 当 $N=1$ 时，序列 $\boldsymbol{x}_{1:T}$ 中每个词都和其他词独立，和它的上下文无关. 每个位置上的词都是从多项分布独立生成的. 在多项分布中，$\theta = [\theta_1, \cdots, \theta_{|\mathcal{V}|}]$ 为词表中每个词被抽取的概率.

多项分布参见第D.2.2.1节.

在一元模型中，序列 $\boldsymbol{x}_{1:T}$ 的概率可以写为

$$p(\boldsymbol{x}_{1:T};\theta) = \prod_{t=1}^{T} p(x_t) = \prod_{k=1}^{|\mathcal{V}|} \theta_k^{m_k}, \tag{15.10}$$

其中 m_k 为词表中第 k 个词 v_k 在序列中出现的次数. 公式(15.10)和标准多项分布的区别是没有多项式系数，因为这里词的顺序是给定的.

给定一组训练集 $\{\boldsymbol{x}_{1:T_n}^{(n)}\}_{n=1}^{N'}$，其对数似然函数为：

$$\log \prod_{n=1}^{N'} p(\boldsymbol{x}_{1:T_n}^{(n)};\theta) = \log \prod_{k=1}^{|\mathcal{V}|} \theta_k^{m_k} \tag{15.11}$$

$$= \sum_{k=1}^{|\mathcal{V}|} m_k \log \theta_k, \tag{15.12}$$

其中 m_k 为第 k 个词在整个训练集中出现的次数.

这样，一元模型的最大似然估计可以转化为约束优化问题：

$$\max_{\theta} \quad \sum_{k=1}^{|\mathcal{V}|} m_k \log \theta_k \tag{15.13}$$

$$\text{s.t.} \quad \sum_{k=1}^{|\mathcal{V}|} \theta_k = 1. \tag{15.14}$$

引入拉格朗日乘子 λ，定义拉格朗日函数 $\Lambda(\theta, \lambda)$ 为

拉格朗日乘子参见第C.3节.

$$\Lambda(\theta, \lambda) = \sum_{k=1}^{|\mathcal{V}|} m_k \log \theta_k + \lambda \left(\sum_{k=1}^{|\mathcal{V}|} \theta_k - 1 \right). \tag{15.15}$$

令

$$\frac{\partial \Lambda(\theta,\lambda)}{\partial \theta_k} = \frac{m_k}{\theta_k} + \lambda = 0, \qquad k = 1,2,\cdots,|\mathcal{V}| \tag{15.16}$$

$$\frac{\partial \Lambda(\theta,\lambda)}{\partial \lambda} = \sum_{k=1}^{|\mathcal{V}|} \theta_k - 1 = 0. \tag{15.17}$$

求解上述方程得到$\lambda = -\sum_{k=1}^{|\mathcal{V}|} m_k$,进一步得到

$$\theta_k = \frac{m_k}{\sum_{k'=1}^{|\mathcal{V}|} m_{k'}} = \frac{m_k}{\bar{m}}, \tag{15.18}$$

其中$\bar{m} = \sum_{k'=1}^{|\mathcal{V}|} m_{k'}$为文档集合的长度. 因此,最大似然估计等价于频率估计.

N元模型 同理,N元模型中的条件概率$p(x_t|\boldsymbol{x}_{(t-N+1):(t-1)})$也可以通过最大似然函数得到

$$p(x_t|\boldsymbol{x}_{(t-N+1):(t-1)}) = \frac{\mathrm{m}(\boldsymbol{x}_{(t-N+1):t})}{\mathrm{m}(\boldsymbol{x}_{(t-N+1):(t-1)})}, \tag{15.19}$$

参见习题15-1.

其中$\mathrm{m}(\boldsymbol{x}_{(t-N+1):t})$为$\boldsymbol{x}_{(t-N+1):t}$在数据集中出现的次数.

N元模型广泛应用于各种自然语言处理问题,如语音识别、机器翻译、拼音输入法、字符识别等. 通过N元模型,我们可以计算一个序列的概率,从而判断该序列是否符合自然语言的语法和语义规则.

平滑技术 N元模型的一个主要问题是数据稀疏问题. 数据稀疏问题在基于统计的机器学习中是一个常见的问题,主要是由于训练样本不足而导致密度估计不准确. 在一元模型中,如果一个词v在训练数据集中不存在,就会导致任何包含v的句子的概率都为0. 同样在N元模型中,当一个N元组合在训练数据集中不存在时,包含这个组合的句子的概率为0.

数据稀疏问题最直接的解决方法就是增加训练数据集的规模,但其边际效益会随着数据集规模的增加而递减. 以自然语言为例,大多数自然语言都服从Zipf定律(Zipf's Law):"在一个给定自然语言数据集中,一个单词出现的频率与它在频率表里的排名成反比. 出现频率最高的单词的出现频率大约是出现频率第二位的单词的2倍,大约是出现频率第三位的单词的3倍."因此,在自然语言中大部分的词都是低频词,很难通过增加数据集来避免数据稀疏问题.

Zipf定律是美国语言学家George K. Zipf提出的实验定律.

数据稀疏问题的一种解决方法是平滑技术(Smoothing),即给一些没有出现的词组合赋予一定先验概率. 平滑技术是N元模型中一项必不可少的技术,比如加法平滑的计算公式为

$$p(x_t|\boldsymbol{x}_{(t-N+1):(t-1)}) = \frac{\mathrm{m}(\boldsymbol{x}_{(t-N+1):t}) + \delta}{\mathrm{m}(\boldsymbol{x}_{(t-N+1):(t-1)}) + \delta|\mathcal{V}|}, \tag{15.20}$$

其中 $\delta \in (0,1]$ 为常数. $\delta = 1$ 时，称为加 1 平滑.

除了加法平滑，还有很多平滑技术，比如 Good-Turing 平滑、Kneser-Ney 平滑等，其基本思想都是增加低频词的频率，而降低高频词的频率.

参见习题15-2.

15.3 深度序列模型

深度序列模型（Deep Sequence Model）是指利用神经网络模型来估计条件概率 $p_\theta(x_t|\boldsymbol{x}_{1:(t-1)})$. 假设一个神经网络 $f(\cdot;\theta)$，其输入为历史信息 $\tilde{h}_t = \boldsymbol{x}_{1:(t-1)}$，输出为词表 $\mathcal{V}$ 中的每个词 $v_k (1 \leq k \leq |\mathcal{V}|)$ 出现的概率，并满足

$$\sum_{k=1}^{|\mathcal{V}|} f_k\big(\boldsymbol{x}_{1:(t-1)};\theta\big) = 1, \tag{15.21}$$

其中 θ 表示网络参数. 条件概率 $p_\theta\big(x_t|\boldsymbol{x}_{1:(t-1)}\big)$ 可以从神经网络的输出中得到：

$$p_\theta\big(x_t|\boldsymbol{x}_{1:(t-1)}\big) = f_{k_{x_t}}\big(\boldsymbol{x}_{1:(t-1)};\theta\big), \tag{15.22}$$

其中 k_{x_t} 为 x_t 在词表 $\mathcal{V}$ 中的索引.

15.3.1 模型结构

深度序列模型一般可以分为三个模块：嵌入层、特征层、输出层.

这里的层泛指神经网络模块（Module），可以由一个或多个神经层组成.

15.3.1.1 嵌入层

令 $\tilde{h}_t = \boldsymbol{x}_{1:(t-1)}$ 表示输入的历史信息，一般为符号序列. 由于神经网络模型一般要求输入形式为实数向量，因此为了使得神经网络模型能处理符号数据，需要将这些符号转换为向量形式. 一种简单的转换方法是通过一个嵌入表（Embedding Lookup Table）来将每个符号直接映射成向量表示. 嵌入表也称为嵌入矩阵或查询表. 图15.2是嵌入矩阵的示例.

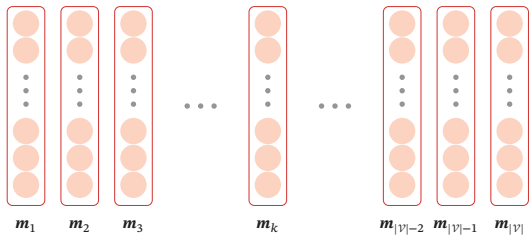


图 15.2 嵌入矩阵

令 $M \in \mathbb{R}^{D_x \times |\mathcal{V}|}$ 为嵌入矩阵，其中第 k 列向量 $\boldsymbol{m}_k \in \mathbb{R}^{D_x}$ 表示词表中第 k 个词对应的向量表示. 假设词 x_t 对应词表中的索引为 k，则其 one-hot 向量表示为 $\delta_t \in \{0,1\}^{|\mathcal{V}|}$，即第 k 维为 1，其余为 0 的 $|\mathcal{V}|$ 维向量. 词 x_t 对应的向量表示为

$$\boldsymbol{e}_t = M\delta_t = \boldsymbol{m}_k. \tag{15.23}$$

通过上面的映射可以得到序列 $x_{1:(t-1)}$ 对应的向量序列 $\boldsymbol{e}_1, \cdots, \boldsymbol{e}_{t-1}$.

15.3.1.2 特征层

特征层用于从输入向量序列 $\boldsymbol{e}_1, \cdots, \boldsymbol{e}_{t-1}$ 中提取特征，输出为一个可以表示历史信息的向量 $\boldsymbol{h}_t$.

特征层可以通过不同类型的神经网络（比如前馈神经网络和循环神经网络等）来实现. 常见的网络类型有以下三种：

（1）简单平均

历史信息的向量 $\boldsymbol{h}_t$ 为前面 $t-1$ 个词向量的平均，即

$$\boldsymbol{h}_t = \sum_{i=1}^{t-1} \alpha_i \boldsymbol{e}_i, \tag{15.24}$$

其中 α_i 为每个词的权重.

注意力机制参见第8.2节.

权重 α_i 可以和位置 i 及其表示 $\boldsymbol{e}_i$ 相关，也可以无关. 为简单起见，可以设置 $\alpha_i = \frac{1}{t-1}$. 权重 α_i 也可以通过注意力机制来动态计算.

参见习题15-3.

（2）前馈神经网络

前馈神经网络要求输入的大小是固定的. 因此，和 N 元模型类似，假设历史信息只包含前面 $N-1$ 个词. 首先将这 $N-1$ 个词向量 $\boldsymbol{e}_{t-N+1}, \cdots, \boldsymbol{e}_{t-1}$ 拼接成一个 $D_x \times (N-1)$ 维的向量 $\boldsymbol{h}'$，即

N 为超参数.

$$\boldsymbol{h}' = \boldsymbol{e}_{t-N+1} \oplus \cdots \oplus \boldsymbol{e}_{t-1}, \tag{15.25}$$

其中 $\oplus$ 表示向量拼接操作.

然后将 $\boldsymbol{h}'$ 输入到由前馈神经网络构成的隐藏层，最后一层隐藏层的输出 $\boldsymbol{h}_t$，即

$$\boldsymbol{h}_t = g(\boldsymbol{h}'; \theta_g), \tag{15.26}$$

其中 $g(\cdot; \theta_g)$ 可以为全连接的前馈神经网络或卷积神经网络，θ_g 为网络参数.

跳层连接是指前馈神经网络中的某一神经层可以接受来自非相邻的低层信息，其思想和残差网络中的直连边并不完全一样. 残差网络参见第5.4.4节.

为了增加特征的多样性和提高模型训练效率，前馈神经网络中也可以包含跳层连接（Skip-Layer Connection）[Bengio et al., 2003]，比如

$$\boldsymbol{h}_t = \boldsymbol{h}' \oplus g(\boldsymbol{h}'; \theta_g). \tag{15.27}$$

（3）循环神经网络

和前馈神经网络不同，循环神经网络可以接受变长的输入序列，依次接受输入 $\boldsymbol{e}_1, \cdots, \boldsymbol{e}_{t-1}$，得到时刻 t 的隐藏状态

$$\boldsymbol{h}_t = g(\boldsymbol{h}_{t-1}, \boldsymbol{e}_t; \theta_g), \tag{15.28}$$

其中 $g(\cdot)$ 为一个非线性函数，θ_g 为循环神经网络的参数，$\boldsymbol{h}_0 = 0$.

前馈神经网络模型和循环神经网络模型的不同之处在于循环神经网络利用隐藏状态来记录以前所有时刻的信息，而前馈神经网络只能接受前 $N-1$ 个时刻的信息.

15.3.1.3 输出层

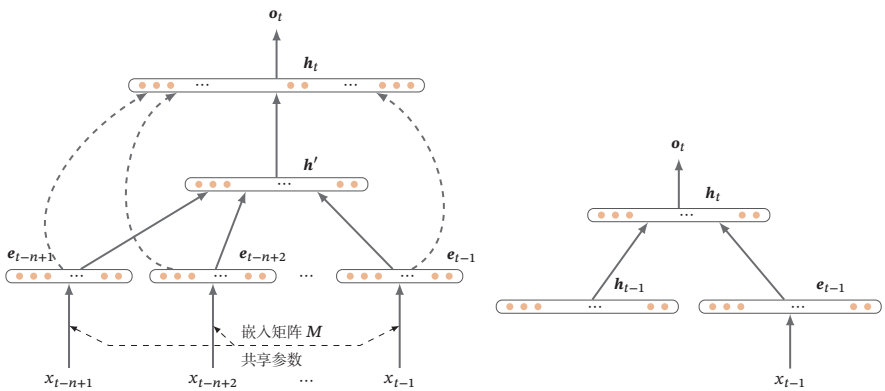
输出层一般使用 Softmax 分类器，接受历史信息的向量表示 $\boldsymbol{h}_t \in \mathbb{R}^{D_h}$，输出为词表中每个词的后验概率，输出大小为 $|\mathcal{V}|$.

$$\boldsymbol{o}_t = \text{softmax}(\hat{\boldsymbol{o}}_t) \tag{15.29}$$

$$= \text{softmax}(\boldsymbol{W}\boldsymbol{h}_t + \boldsymbol{b}), \tag{15.30}$$

其中输出向量 $\boldsymbol{o}_t \in (0,1)^{|\mathcal{V}|}$ 为预测的概率分布，第 k 维是词表中第 k 个词出现的条件概率；$\hat{\boldsymbol{o}}_t$ 是未归一化的得分向量；$\boldsymbol{W} \in \mathbb{R}^{|\mathcal{V}| \times D_h}$ 是最后一层隐藏层到输出层直接的权重矩阵，$\boldsymbol{b} \in \mathbb{R}^{|\mathcal{V}|}$ 为偏置.

图15.3给出了两种不同的深度序列模型，图15.3a为前馈神经网络模型（虚线边为可选的跳层连接），图15.3b为循环神经网络模型.



(a) 前馈神经网络模型 (b) 循环神经网络模型

图 15.3 深度序列模型

15.3.2 参数学习

给定一个训练序列 $\boldsymbol{x}_{1:T}$，深度序列模型的训练目标是找到一组参数 θ 使得对数似然函数最大.

简要起见，这里忽略了正则化项.

$$\log p_\theta(\boldsymbol{x}_{1:T}) = \sum_{t=1}^{T} \log p_\theta\big(x_t|\boldsymbol{x}_{1:(t-1)}\big), \tag{15.31}$$

其中 θ 表示网络中的所有参数，包括嵌入矩阵 M 以及神经网络的权重和偏置.

网络参数一般通过梯度上升法来学习，

$$\theta \leftarrow \theta + \alpha \frac{\partial \log p_\theta(\boldsymbol{x}_{1:T})}{\partial \theta}, \tag{15.32}$$

其中 α 为学习率.

15.4 评价方法

构造一个序列生成模型后，需要有一个度量来评价其好坏.

15.4.1 困惑度

给定一个测试文本集合，一个好的序列生成模型应该使得测试集合中句子的联合概率尽可能高.

困惑度（Perplexity）是信息论中的一个概念，可以用来衡量一个分布的不确定性. 对于离散随机变量 $X \in \mathcal{X}$，其概率分布为 $p(x)$，困惑度为

$$2^{\boldsymbol{H}(p)} = 2^{-\sum_{x\in\mathcal{X}} p(x)\log_2 p(x)}, \tag{15.33}$$

其中 $\boldsymbol{H}(p)$ 为分布 p 的熵.

困惑度也可以用来衡量两个分布之间差异. 对于一个未知的数据分布 $p_r(x)$ 和一个模型分布 $p_\theta(x)$，我们从 $p_r(x)$ 中采样出一组测试样本 $x^{(1)}, \cdots, x^{(N)}$，模型分布 $p_\theta(x)$ 的困惑度为

$$2^{\boldsymbol{H}(\tilde{p}_r, p_\theta)} = 2^{-\frac{1}{N}\sum_{n=1}^{N} \log_2 p_\theta(x^{(n)})}, \tag{15.34}$$

其中 $\boldsymbol{H}(\tilde{p}_r, p_\theta)$ 为样本的经验分布 $\tilde{p}_r$ 与模型分布 p_θ 之间的交叉熵，也是所有样本上的负对数似然函数.

困惑度可以衡量模型分布与样本经验分布之间的契合程度. 困惑度越低则两个分布越接近. 因此，模型分布 $p_\theta(x)$ 的好坏可以用困惑度来评价.

假设测试集合有 N 个独立同分布的序列 $\{\boldsymbol{x}_{1:T_n}^{(n)}\}_{n=1}^{N}$. 我们可以用模型 $p_\theta(\boldsymbol{x})$ 对每个序列计算其概率 $p_\theta(\boldsymbol{x}_{1:T_n}^{(n)})$, 整个测试集的联合概率为

$$\prod_{n=1}^{N} p_\theta(\boldsymbol{x}_{1:T_n}^{(n)}) = \prod_{n=1}^{N}\prod_{t=1}^{T_n} p_\theta(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)}). \tag{15.35}$$

模型 $p_\theta(\boldsymbol{x})$ 的困惑度定义为

$$\begin{aligned}\mathrm{PPL}(\theta) &= 2^{-\frac{1}{T}\sum_{n=1}^{N}\log_2 p_\theta\left(\boldsymbol{x}_{1:T_n}^{(n)}\right)} &(15.36)\\ &= 2^{-\frac{1}{T}\sum_{n=1}^{N}\sum_{t=1}^{T_n}\log_2 p_\theta\left(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)}\right)} &(15.37)\\ &= \left(\prod_{n=1}^{N}\prod_{t=1}^{T_n} p_\theta\left(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)}\right)\right)^{-1/T}, &(15.38)\end{aligned}$$

其中 $T = \sum_{n=1}^{N} T_n$ 为测试数据集中序列的总长度. 可以看出, 困惑度为每个词条件概率 $p_\theta(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)})$ 的几何平均数的倒数. 测试集中所有序列的概率越大, 困惑度越小, 模型越好.

几何平均数是一种求数值平均数的方法, 计算公式为: $\bar{x} = \sqrt[n]{\prod_{t=1}^{n} x_t}$.

假设一个序列模型赋予每个词出现的概率均等, 即 $p_\theta(x_t^{(n)}|\boldsymbol{x}_{1:(t-1)}^{(n)}) = \frac{1}{|\mathcal{V}|}$, 则该模型的困惑度为 $|\mathcal{V}|$. 以英语为例, N 元模型的困惑度范围一般为 $50 \sim 1000$.

15.4.2 BLEU 算法

BLEU (BiLingual Evaluation Understudy) 算法是一种衡量模型生成序列和参考序列之间的 N 元词组 (N-Gram) 重合度的算法, 最早用来评价机器翻译模型的质量, 目前也广泛应用在各种序列生成任务中.

令 $\boldsymbol{x}$ 为从模型分布 p_θ 中生成的一个候选 (Candidate) 序列, $\boldsymbol{s}^{(1)}, \cdots, \boldsymbol{s}^{(K)}$ 为从真实数据分布中采集的一组参考 (Reference) 序列, $\mathcal{W}$ 为从生成的候选序列中提取所有N 元组合的集合, 这些 N 元组合的精度 (Precision)

$$P_N(\boldsymbol{x}) = \frac{\sum_{w\in\mathcal{W}} \min\left(c_w(\boldsymbol{x}), \max_{k=1}^{K} c_w(\boldsymbol{s}^{(k)})\right)}{\sum_{w\in\mathcal{W}} c_w(\boldsymbol{x})}, \tag{15.39}$$

其中 $c_w(\boldsymbol{x})$ 是 N 元组合 w 在生成序列 $\boldsymbol{x}$ 中出现的次数, $c_w(\boldsymbol{s}^{(k)})$ 是 N 元组合 w 在参考序列 $\boldsymbol{s}^{(k)}$ 中出现的次数. N 元组合的精度 $P_N(\boldsymbol{x})$ 是计算生成序列中的 N 元组合有多少比例在参考序列中出现.

由于精度只衡量生成序列中的 N 元组合是否在参考序列中出现, 生成序列越短, 其精度会越高, 因此可以引入长度惩罚因子 (Brevity Penalty). 如果生成序

列的长度短于参考序列，就对其进行惩罚.

$$b(\boldsymbol{x}) = \begin{cases} 1 & \text{if} \quad l_x > l_s \\ \exp\left(1 - l_s/l_x\right) & \text{if} \quad l_x \leq l_s \end{cases} \tag{15.40}$$

其中 l_x 为生成序列 $\boldsymbol{x}$ 的长度，l_s 为参考序列的最短长度.

BLEU 算法是通过计算不同长度的 N 元组合（$N = 1, 2, \cdots$）的精度，并进行几何加权平均而得到.

$$\text{BLEU-N}(\boldsymbol{x}) = b(\boldsymbol{x}) \times \exp\Big(\sum_{N=1}^{N'} \alpha_N \log P_N\Big), \tag{15.41}$$

其中 N' 为最长 N 元组合的长度，α_N 为不同 N 元组合的权重，一般设为 $\frac{1}{N'}$. BLEU 算法的值域范围是 $[0, 1]$，越大表明生成的质量越好. 但是 BLEU 算法只计算精度，而不关心召回率（即参考序列里的 N 元组合是否在生成序列中出现）.

参见习题15-4.

15.4.3 ROUGE 算法

ROUGE（Recall-Oriented Understudy for Gisting Evaluation）算法最早应用于文本摘要领域. 和 BLEU 算法类似，但 ROUGE 算法计算的是召回率（Recall）.

令 $\boldsymbol{x}$ 为从模型分布 p_θ 中生成的一个候选序列，$\boldsymbol{s}^{(1)}, \cdots, \boldsymbol{s}^{(K)}$ 为从真实数据分布中采样出的一组参考序列，$\mathcal{W}$ 为从参考序列中提取 N 元组合的集合，ROUGE-N 算法的定义为

$$\text{ROUGE-N}(\boldsymbol{x}) = \frac{\sum_{k=1}^{K} \sum_{w \in \mathcal{W}} \min\left(c_w(\boldsymbol{x}), c_w(\boldsymbol{s}^{(k)})\right)}{\sum_{k=1}^{K} \sum_{w \in \mathcal{W}} c_w(\boldsymbol{s}^{(k)})}, \tag{15.42}$$

其中 $c_w(\boldsymbol{x})$ 是 N 元组合 w 在生成序列 $\boldsymbol{x}$ 中出现的次数，$c_w(\boldsymbol{s}^{(k)})$ 是 N 元组合 w 在参考序列 $\boldsymbol{s}^{(k)}$ 中出现的次数.

15.5 序列生成模型中的学习问题

使用最大似然估计来学习自回归序列生成模型时，会存在以下三个主要问题：曝光偏差问题、训练目标不一致问题和计算效率问题. 下面我们分别介绍这三个问题以及解决方法.

15.5.1 曝光偏差问题

在自回归生成模型中，第 t 步的输入为模型生成的前缀序列 $\hat{\boldsymbol{x}}_{1:(t-1)}$. 而在训练时，我们使用的前缀序列是训练集中的真实数据 $\boldsymbol{x}_{1:(t-1)}$，而不是模型预测的 $\hat{\boldsymbol{x}}_{1:(t-1)}$. 这种学习方式也称为教师强制（Teacher Forcing）[Williams et al., 1989].

这种教师强制的学习方式存在协变量偏移问题. 在训练时，每步的输入 $\boldsymbol{x}_{1:(t-1)}$ 来自于真实数据分布 $p_r(\boldsymbol{x}_{1:(t-1)})$；而在测试时，每步的输入 $\hat{\boldsymbol{x}}_{1:(t-1)}$ 来自于模型分布 $p_\theta(\boldsymbol{x}_{1:(t-1)})$. 由于模型分布 $p_\theta(\boldsymbol{x}_{1:(t-1)})$ 和真实数据分布 $p_r(\boldsymbol{x}_{1:(t-1)})$ 并不严格一致，因此存在协变量偏移问题. 一旦在预测前缀 $\hat{\boldsymbol{x}}_{1:(t-1)}$ 的过程中存在错误，会导致错误传播，使得后续生成的序列也会偏离真实分布. 这个问题称为曝光偏差（Exposure Bias）问题.

协变量偏移问题参见第10.4.2节.

计划采样 为了缓解曝光偏差问题，我们可以在训练时混合使用真实数据和模型生成数据. 在第 t 步时，模型随机使用真实数据 x_{t-1} 或前一步生成的词 $\hat{x}_{t-1}$ 作为输入.

令 $\epsilon \in [0,1]$ 为一个控制替换率的超参数，在每一步时，以 ϵ 的概率使用真实数据 x_{t-1}，以 $1-\epsilon$ 的概率使用生成数据 $\hat{x}_{t-1}$. 当令 $\epsilon = 1$ 时，训练和最大似然估计一样，使用真实数据；当令 $\epsilon = 0$ 时，训练时全部使用模型生成数据.

直觉上，如果一开始训练时的 ϵ 过小，模型相当于在噪声很大的数据上训练，会导致模型性能变差，并且难以收敛. 因此，一个较好的策略是在训练初期赋予 ϵ 较大的值，随着训练次数的增加逐步减小 ϵ 的取值. 这种策略称为计划采样（Scheduled Sampling）[Bengio et al., 2015].

令 ϵ_i 为在第 i 次迭代时的替换率，在计划采样中可以通过下面几种方法来逐步减小 ϵ 的取值.

（1）线性衰减：$\epsilon_i = \max(\epsilon, k - ci)$，其中 ϵ 为最小的替换率，k 和 c 分别为初始值和衰减率.

（2）指数衰减：$\epsilon_i = k^i$，其中 $k < 1$ 为初始替换率.

（3）逆 Sigmoid 衰减：$\epsilon_i = k/(k + \exp(i/k))$，其中 $k \geq 1$ 来控制衰减速度.

计划采样的一个缺点是过度纠正，即在每一步中不管输入如何选择，目标输出依然是来自于真实数据. 这可能使得模型预测一些不正确的序列. 比如一个真实的序列是“吃饭”，如果在第一步生成时使用模型预测的词是“喝”，模型就会强制记住“喝饭”这个不正确的序列.

15.5.2 训练目标不一致问题

序列生成模型的好坏通常采用和任务相关的指标来进行评价，比如 BLEU、ROUGE 等. 然而，在训练时通常是使用最大似然估计来优化模型，这导致训练目标和评价方法不一致. 并且这些评价指标一般都是不可微的，无法直接使用基于梯度的方法来进行优化.

基于强化学习的序列生成 为了可以直接优化评价目标，我们可以将自回归序列生成看作一种马尔可夫决策过程，并使用强化学习的方法来进行训练.

参见第14.1.3节.

在第 t 步，动作 a_t 可以看作从词表中选择一个词，策略为 $\pi_\theta(a|s_t)$，其中状态 s_t 为之前步骤中生成的前缀序列 $\boldsymbol{x}_{1:(t-1)}$. 我们可以把一个序列 $\boldsymbol{x}_{1:T}$ 看作马尔可夫决策过程的一个轨迹（trajectory）：

$$\tau = \{s_1, a_1, s_2, a_2, ..., s_T, a_T\}. \tag{15.43}$$

轨迹 τ 的概率为

$$p_\theta(\tau) = \prod_{t=1}^{T} \pi_\theta\big(a_t = x_t | s_t = \boldsymbol{x}_{1:(t-1)}\big), \tag{15.44}$$

其中状态转移概率 $p\big(s_t = \boldsymbol{x}_{1:t-1} | s_{t-1} = \boldsymbol{x}_{1:(t-2)}, a_{t-1} = x_{t-1}\big) = 1$ 是确定性的，可以被忽略.

强化学习的目标是学习一个策略 $\pi_\theta(a|s_t)$ 使得期望回报最大，

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[G(\tau)] \tag{15.45}$$

$$= \mathbb{E}_{\boldsymbol{x}_{1:T} \sim p_\theta(\boldsymbol{x}_{1:T})}[G(\boldsymbol{x}_{1:T})], \tag{15.46}$$

其中 $G(\boldsymbol{x}_{1:T})$ 为序列 $\boldsymbol{x}_{1:T}$ 的总回报，可以为 BLEU、ROUGE 或其他评价指标.

这样，序列生成问题就转换为强化学习问题，其策略函数 $\pi_\theta(a|s_t)$ 可以通过 REINFORCE 算法或演员-评论员算法来进行学习. 为了改进强化学习的效率，策略函数 $\pi_\theta(a|s_t)$ 一般会通过最大似然估计来进行预训练.

基于强化学习的序列生成模型不但可以解决训练和评价目标不一致问题，也可以有效地解决曝光偏差问题.

15.5.3 计算效率问题

序列生成模型的输出层为词表中所有词的条件概率，需要 Softmax 归一化. 当词表比较大时，计算效率比较低.

在第 t 步时，前缀序列为 $\tilde{h}_t = \boldsymbol{x}_{1:(t-1)}$，词 x_t 的条件概率为

$$p_\theta(x_t|\tilde{h}_t) = \text{softmax}\Big(s(x_t, \tilde{h}_t; \theta)\Big) \tag{15.47}$$

$s(x_t, \tilde{h}_t; \theta) = [\hat{\boldsymbol{o}}_t]_{k_{x_t}}$，$\hat{\boldsymbol{o}}_t$ 为未归一化的网络输出，k_{x_t} 为 x_t 在词表 $\mathcal{V}$ 中的索引，参见公式(15.29).

$$= \frac{\exp\left(s(x_t, \tilde{h}_t; \theta)\right)}{\sum_{v \in \mathcal{V}} \exp\left(s(v, \tilde{h}_t; \theta)\right)}, \tag{15.48}$$

$$= \frac{\exp\left(s(x_t, \tilde{h}_t; \theta)\right)}{Z(\tilde{h}_t; \theta)}, \tag{15.49}$$

其中 $s(x_t, \tilde{h}_t; \theta)$ 为未经过 Softmax 归一化的得分函数，$Z(\tilde{h}_t; \theta)$ 为配分函数（Partition Function）：

配分函数参见第11.1.4节.

$$Z(\tilde{h}_t; \theta) = \sum_{v \in \mathcal{V}} \exp\left(s(v, \tilde{h}_t; \theta)\right). \tag{15.50}$$

配分函数的计算需要对词表中所有的词 v 计算 $s(v, \tilde{h}_t; \theta)$ 并求和. 当词表比较大时，计算开销非常大. 比如在自然语言中，词表 $\mathcal{V}$ 的规模一般在 1 万到 10 万之间. 在训练时，每个样本都要计算一次配分函数，这样每一轮迭代需要计算 T 次配分函数（T 为训练文本长度），导致整个训练过程变得十分耗时. 因此在实践中，我们通常采用一些近似估计的方法来加快训练速度. 常用的方法可以分为两类：1）层次化的 Softmax 方法，将标准 Softmax 函数的扁平结构转换为层次化结构；2）基于采样的方法，通过采样来近似计算更新梯度.

本节介绍三种加速训练速度的方法：层次化 Softmax、重要性采样和噪声对比估计.

15.5.3.1 层次化 Softmax

我们先来考虑使用两层的树结构来组织词表，即将词表中的词分成 K 组，并且每一个词只能属于一个分组，每组大小为 $\frac{|\mathcal{V}|}{K}$. 假设词 w 所属的组为 $c(w)$，则

$$p(w|\tilde{h}) = p(w, c(w)|\tilde{h}) \tag{15.51}$$

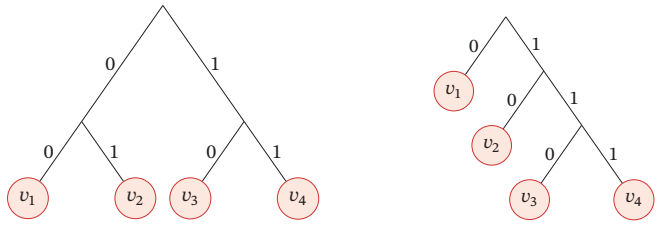
$$= p(w|c(w), \tilde{h}) p(c(w)|\tilde{h}), \tag{15.52}$$

其中 $p(c(w)|\tilde{h})$ 是给定历史信息 $\tilde{h}$ 条件下，类 $c(w)$ 的后验概率，$p(w|c(w), \tilde{h})$ 是给定历史信息 $\tilde{h}$ 和类 $c(w)$ 条件下，词 w 的后验概率. 因此，一个词的概率可以分解为两个概率 $p(w|c(w), \tilde{h})$ 和 $p(c(w)|\tilde{h})$ 的乘积，它们可以分别利用神经网络来估计，这样计算 Softmax 函数时分别只需要做 $\frac{|\mathcal{V}|}{K}$ 和 K 次求和，从而大大提高了 Softmax 函数的计算速度.

一般对于词表大小 $|\mathcal{V}|$，我们将词平均分到 $\sqrt{|\mathcal{V}|}$ 个分组中，每组 $\sqrt{|\mathcal{V}|}$ 个词. 这样通过一层的分组，我们可以将 Softmax 计算加速 $\frac{1}{2}\sqrt{|\mathcal{V}|}$ 倍. 比如当词表大小为 40,000 时，将词表中所有词分到 200 组，每组 200 个词. 这样只需要计算两次 200 类的 Softmax，比直接计算 40,000 类的 Softmax 加快 100 倍.

为了进一步降低 Softmax 函数的计算复杂度，我们可以使用更深层的树结构来组织词汇表. 假设用二叉树来组织词表中的所有词，二叉树的叶子节点代表

词表中的词，非叶子节点表示不同层次上的类别. 图15.4给出了平衡二叉树和霍夫曼编码二叉树的示例.



(a) 平衡树 (b) 霍夫曼编码树

图 15.4 层次化树结构

如果我们将二叉树上所有左连边标记为 0，右连边标记为 1. 每一个词可以用根节点到它所在的叶子之间路径上的标记来进行编码. 图15.4a中所示的四个词的编码分别为：

$$v_1 = 00, \qquad v_2 = 01, \qquad v_3 = 10, \qquad v_4 = 11. \tag{15.53}$$

假设词 v 在二叉树上从根节点到其所在叶子节点的路径长度为 M，其编码可以表示一个位向量（bit vector）：$[b_1, \cdots, b_M]^{\mathrm{T}}$. 词 v 的条件概率为

$$P(v|\tilde{h}) = p(b_1, \cdots, b_M|\tilde{h}) \tag{15.54}$$

$$= \prod_{m=1}^{M} p(b_m|b_1, \cdots, b_{m-1}, \tilde{h}), \tag{15.55}$$

$$= \prod_{m=1}^{M} p(b_m|b_{m-1}, \tilde{h}). \tag{15.56}$$

由于 $b_m \in \{0, 1\}$ 为二值变量，可以将 $p(b_m|b_{m-1}, \tilde{h})$ 看作二分类问题，使用 Logistic 回归来进行预测：

$$p(b_m = 1|b_{m-1}, \tilde{h}) = \sigma\big(\boldsymbol{w}_{n(b_{m-1})}^{\mathrm{T}} \boldsymbol{h} + \boldsymbol{b}_{n(b_{m-1})}\big), \tag{15.57}$$

其中 $n(b_{m-1})$ 为词 v 在树 T 上的路径上的第 $m-1$ 个节点.

若使用平衡二叉树来进行分组，则条件概率估计可以转换为 $\log_2 |\mathcal{V}|$ 个二分类问题. 这时原始预测模型中的 Softmax 函数可以用 Logistic 函数代替，计算效率可以加速 $\frac{|\mathcal{V}|}{\log_2 |\mathcal{V}|}$ 倍.

将词表中的词按照树结构进行组织，有以下几种转换方式：

（1）利用人工整理的词汇层次结构，比如利用 WordNet[Miller, 1995] 系统中的“IS-A”关系（即上下位关系）. 例如，“狗”是“动物”的下位词. 因为 WordNet 的层次化结构不是二叉树，因此需要通过进一步聚类来转换为二叉树.

WordNet 是按照词义来组织的英语词汇知识库，由普林斯顿大学研发.

（2）使用霍夫曼编码（Huffman Coding）. 霍夫曼编码对出现概率高的词使用较短的编码，出现概率低的词使用较长的编码. 因此训练速度会更快. 霍夫曼编码的算法如算法15.1所示.

霍夫曼编码是 David Huffman 于 1952 年发明的一种用于无损数据压缩的熵编码算法. 熵编码参见第E.1.2节.

算法 15.1 霍夫曼编码树构建算法

输入: 词表: $\mathcal{V}$

初始化：为每个词 v 建立一个叶子节点，其概率为词的出现频率;
将所有的叶子节点放入集合 $\mathcal{S}$ 中;
while $|\mathcal{S}| > 1$ **do**
  从集合 $\mathcal{S}$ 中选择两个概率最低的节点 n_1 和 n_2;
  构建一个新节点 n'，并将 n_1 和 n_2 作为 n' 的左右子节点;
  新节点 n' 的概率为 n_1 和 n_2 的概率之和;
  将新节点 n' 加入集合 $\mathcal{S}$ 中，并把 n_1 和 n_2 从集合 $\mathcal{S}$ 中移除;
end
集合 $\mathcal{S}$ 中最后一个节点为 n;

输出: 以 n 为根节点的二叉树 T

15.5.3.2 重要性采样

另一种提高训练效率的方法是基于采样的方法，即通过采样来近似计算训练时的梯度.

参见第11.5节.

用随机梯度上升来更新参数 θ 时，第 t 个样本 $(\tilde{h}_t, x_t)$ 的目标函数关于 θ 的梯度为

$$\frac{\partial \log p_\theta(x_t|\tilde{h}_t)}{\partial \theta} = \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \frac{\partial \log\left(\sum_v \exp\left(s(v, \tilde{h}_t; \theta)\right)\right)}{\partial \theta} \tag{15.58}$$

$$= \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \frac{1}{\sum_v \exp\left(s(v, \tilde{h}_t; \theta)\right)} \frac{\partial \sum_v \exp\left(s(v, \tilde{h}_t; \theta)\right)}{\partial \theta} \tag{15.59}$$

$$= \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \sum_v \frac{1}{\sum_w \exp\left(s(w, \tilde{h}_t; \theta)\right)} \frac{\partial \exp\left(s(v, \tilde{h}_t; \theta)\right)}{\partial \theta} \tag{15.60}$$

$$= \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \sum_v \frac{\exp\left(s(v, \tilde{h}_t; \theta)\right)}{\sum_w \exp\left(s(w, \tilde{h}_t; \theta)\right)} \frac{\partial s(v, \tilde{h}_t; \theta)}{\partial \theta} \tag{15.61}$$

$$= \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \sum_v p_\theta(v|\tilde{h}_t) \frac{\partial s(v, \tilde{h}_t; \theta)}{\partial \theta} \tag{15.62}$$

$$= \frac{\partial s(x_t, \tilde{h}_t; \theta)}{\partial \theta} - \mathbb{E}_{p_\theta(v|\tilde{h}_t)} \left[\frac{\partial s(v, \tilde{h}_t; \theta)}{\partial \theta}\right]. \tag{15.63}$$

公式(15.63)中最后一项是计算 $\frac{\partial}{\partial\theta}s(v,\tilde{h}_t;\theta)$ 在分布 $p_\theta(v|\tilde{h}_t)$ 下的期望. 从公式(15.61)中可以看出, 在计算每个样本的梯度时需要在整个词表上计算两次求和. 一次是求配分函数 $\sum_w \exp\big(s(w,\tilde{h}_t;\theta)\big)$, 另一次是计算所有词的梯度的期望 $\mathbb{E}[\frac{\partial}{\partial\theta}s(v,\tilde{h}_t;\theta)]$. 由于自然语言中的词表都比较大, 训练速度会非常慢.

为了提高训练效率, 可以用采样方法来近似地估计公式(15.63)中的期望. 但是我们不能直接根据分布 $p_\theta(v|\tilde{h}_t)$ 进行采样, 因为直接采样需要先计算分布 $p_\theta(v|\tilde{h}_t)$, 而这正是我们希望避免的.

重要性采样是用一个容易采样的提议分布q来近似估计分布 p.

重要性采样参见第11.5.3节.

公式(15.63)中最后一项可以写为

$$\mathbb{E}_{p_\theta(v|\tilde{h}_t)}\left[\frac{\partial s(v,\tilde{h}_t;\theta)}{\partial\theta}\right]=\sum_{v\in\mathcal{V}}p_\theta(v|\tilde{h}_t)\frac{\partial s(v,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.64}$$

$$=\sum_{v\in\mathcal{V}}q(v|\tilde{h}_t)\frac{p_\theta(v|\tilde{h}_t)}{q(v|\tilde{h}_t)}\frac{\partial s(v,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.65}$$

$$=\mathbb{E}_{q(v|\tilde{h}_t)}\left[\frac{p_\theta(v|\tilde{h}_t)}{q(v|\tilde{h}_t)}\frac{\partial s(v,\tilde{h}_t;\theta)}{\partial\theta}\right]. \tag{15.66}$$

这样, 原始分布 $p_\theta(v|\tilde{h}_t)$ 上的期望转换为提议分布 $q(v|\tilde{h}_t)$ 上的期望. 提议分布 q 需要尽可能和 $p_\theta(v|\tilde{h}_t)$ 接近, 并且从 $q(v|\tilde{h}_t)$ 采样的代价要比较小. 在实践中, 提议分布 $q(v|\tilde{h}_t)$ 可以采用 N 元模型的分布函数.

根据分布 $q(v|\tilde{h}_t)$ 独立采样 K 个样本 $v_1,\cdots,v_K$ 来近似求解公式(15.66), 即

$$\mathbb{E}_{p_\theta(v|\tilde{h}_t)}\left[\frac{\partial s(v,\tilde{h}_t;\theta)}{\partial\theta}\right]\approx\frac{1}{K}\sum_{k=1}^{K}\frac{p_\theta(v_k|\tilde{h}_t)}{q(v_k|\tilde{h}_t)}\frac{\partial s(v_k,\tilde{h}_t;\theta)}{\partial\theta}. \tag{15.67}$$

在公式(15.67)中, 依然需要计算每一个抽取样本的概率 $p_\theta(v_k|\tilde{h}_t)$, 即

$$p_\theta(v_k|\tilde{h}_t)=\frac{s(v_k,\tilde{h}_t;\theta)}{Z(\tilde{h}_t)}, \tag{15.68}$$

其中配分函数 $Z(\tilde{h}_t)=\sum_w \exp\big(s(w,\tilde{h}_t;\theta)\big)$ 需要在所有样本上计算 $s(w,\tilde{h}_t;\theta)$ 并求和. 为了避免这种情况, 我们也使用重要性采样来计算配分函数 $Z(\tilde{h}_t)$, 即

$$Z(\tilde{h}_t)=\sum_w \exp\big(s(w,\tilde{h}_t;\theta)\big) \tag{15.69}$$

$$=\sum_w q(w|\tilde{h}_t)\frac{1}{q(w|\tilde{h}_t)}\exp\big(s(w,\tilde{h}_t;\theta)\big) \tag{15.70}$$

$$=\mathbb{E}_{q(w|\tilde{h}_t)}\left[\frac{1}{q(w|\tilde{h}_t)}\exp\big(s(w,\tilde{h}_t;\theta)\big)\right] \tag{15.71}$$

$$\approx\frac{1}{K}\sum_{k=1}^{K}\frac{1}{q(v_k|\tilde{h}_t)}\exp\big(s(v_k,\tilde{h}_t;\theta)\big) \tag{15.72}$$

通过采样近似估计.

$$= \frac{1}{K}\sum_{k=1}^{K}\frac{\exp\left(s(\upsilon_k,\tilde{h}_t;\theta)\right)}{q(\upsilon_k|\tilde{h}_t)} \tag{15.73}$$

$$= \frac{1}{K}\sum_{k=1}^{K} r(\upsilon_k), \tag{15.74}$$

其中 $r(\upsilon_k) = \frac{\exp\left(s(\upsilon_k,\tilde{h}_t;\theta)\right)}{q(\upsilon_k|\tilde{h}_t)}$，$q(\upsilon_k|\tilde{h}_t)$ 为提议分布. 为了提高效率，可以和公式(15.67)中的提议分布设为一致，并复用在上一步中抽取的样本.

在近似估计了配分函数以及梯度期望之后，公式(15.67)可写为

$$\mathbb{E}_{p_\theta(\upsilon|\tilde{h}_t)}\left[\frac{\partial s(\upsilon,\tilde{h}_t;\theta)}{\partial\theta}\right] \approx \frac{1}{K}\sum_{k=1}^{K}\frac{p_\theta(\upsilon_k|\tilde{h}_t)}{q(\upsilon_k|\tilde{h}_t)}\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.75}$$

$$= \frac{1}{K}\sum_{k=1}^{K}\frac{\exp\left(s(\upsilon_k,\tilde{h}_t;\theta)\right)}{Z(\tilde{h}_t)}\frac{1}{q(\upsilon_k|\tilde{h}_t)}\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.76}$$

$$= \frac{1}{K}\sum_{k=1}^{K}\frac{1}{Z(\tilde{h}_t)}r(\upsilon_k)\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.77}$$

$$\approx \sum_{k=1}^{K}\frac{r(\upsilon_k)}{\sum_{k=1}^{K}r(\upsilon_k)}\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta} \tag{15.78}$$

$$= \frac{1}{\sum_{k=1}^{K}r(\upsilon_k)}\sum_{k=1}^{K}r(\upsilon_k)\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta}. \tag{15.79}$$

将公式(15.79)代入公式(15.63)，得到每个样本目标函数关于 θ 的梯度可以近似为

$$\frac{\partial\log p_\theta(x_t|\tilde{h}_t)}{\partial\theta} = \frac{\partial s(x_t,\tilde{h}_t;\theta)}{\partial\theta} - \frac{1}{\sum_{k=1}^{K}r(\upsilon_k)}\sum_{k=1}^{K}r(\upsilon_k)\frac{\partial s(\upsilon_k,\tilde{h}_t;\theta)}{\partial\theta}, \tag{15.80}$$

其中 $\upsilon_1,\cdots,\upsilon_K$ 是根据提议分布 $q(\upsilon|\tilde{h}_t)$ 从词表 $\mathcal{V}$ 中采样的词. 和公式(15.63)相比，重要性采样相当于采样了一个词表的子集 $\mathcal{V}' = \{\upsilon_1,\cdots,\upsilon_K\}$，然后在这个子集上求梯度 $\frac{\partial s(\upsilon_k,\tilde{h};\theta)}{\partial\theta}$ 的期望；公式(15.63)中分布 $p_\theta(\upsilon|\tilde{h}_t)$ 被 $r(\upsilon_k)$ 所替代. 这样目标函数关于 θ 的梯度就避免了在词表上对所有词进行计算，只需要计算较少的抽取的样本. 采样的样本数量 K 越大，近似越接近正确值. 在实际应用中，K 取 100 左右就能够以足够高的精度对期望做出估计. 通过重要性采样的方法，训练速度可以加速 $\frac{|\mathcal{V}|}{K}$ 倍.

重要性采样的思想和算法都比较简单，但其效果依赖于提议分布 $q(\upsilon|\tilde{h}_t)$ 的选取. 如果 $q(\upsilon|\tilde{h}_t)$ 选取不合适，会造成梯度估计非常不稳定. 在实践中，提议分布 $q(\upsilon|\tilde{h}_t)$ 经常使用一元模型的分布函数. 虽然直观上 $q(\upsilon|\tilde{h}_t)$ 采用 N 元模型更加

准确，但使用复杂的 N 元模型分布并不能改进性能，原因是 N 元模型的分布和神经网络模型估计的分布之间有很大的差异 [Bengio et al., 2008].

15.5.3.3 噪声对比估计

除重要性采样外，噪声对比估计（Noise-Contrastive Estimation，NCE）也是一种常用的近似估计梯度的方法.

噪声对比估计是将密度估计问题转换为二分类问题，从而降低计算复杂度 [Gutmann et al., 2010]. 噪声对比估计的思想在我们日常生活中十分常见. 比如我们教小孩认识“苹果”，往往会让小孩从一堆各式各样的水果中找出哪个是“苹果”. 通过不断的对比和纠错，最终小孩会知道“苹果”的特征，并很容易识别出“苹果”.

噪声对比估计的数学描述如下：假设有三个分布，第一个是需要建模的真实数据分布 $p_r(x)$；第二个是模型分布 $p_\theta(x)$，并期望调整模型参数 θ 使得 $p_\theta(x)$ 可以拟合真实数据分布 $p_r(x)$；第三个是噪声分布 $q(x)$，用来对比学习. 给定一个样本 x，如果 x 是从 $p_r(x)$ 中抽取的，则称为真实样本，如果 x 是从 $q(x)$ 中抽取的，则称为噪声样本. 为了判断样本 x 是真实样本还是噪声样本，引入一个判别函数 D.

噪声对比估计是通过调整模型 $p_\theta(x)$ 使得判别函数 D 很容易分辨出样本 x 来自哪个分布. 令 $y \in \{1, 0\}$ 表示一个样本 x 是真实样本或噪声样本，其条件概率为

$$p(x|y=1) = p_\theta(x), \tag{15.81}$$

$$p(x|y=0) = q(x). \tag{15.82}$$

一般噪声样本的数量要比真实样本大很多. 为了提高近似效率，我们近似假设噪声样本的数量是真实样本的 K 倍，即 y 的先验分布满足

$$p(y=0) = Kp(y=1). \tag{15.83}$$

根据贝叶斯公式，样本 x 来自于真实数据分布的后验概率为

$$p(y=1|x) = \frac{p(x|y=1)p(y=1)}{p(x|y=1)p(y=1) + p(x|y=0)p(y=0)} \tag{15.84}$$

$$= \frac{p_\theta(x)p(y=1)}{p_\theta(x)p(y=1) + q(x)kp(y=1)} \tag{15.85}$$

$$= \frac{p_\theta(x)}{p_\theta(x) + Kq(x)}. \tag{15.86}$$

相反，样本 x 来自于噪声分布的后验概率为 $p(y=0|x) = 1 - p(y=1|x)$.

从真实分布 $p_r(x)$ 中抽取 N 个样本 $x_1, \cdots, x_N$，将其类别设为 $y=1$，然后从噪声分布中抽取 KN 个样本 $x'_1, \cdots, x'_{KN}$，将其类别设为 $y=0$. 噪声对比估计的

目标是将真实样本和噪声样本区别开来，可以看作一个二分类问题. 噪声对比估计的损失函数为

$$\mathcal{L}(\theta) = -\frac{1}{N(K+1)}\left(\sum_{n=1}^{N}\log p(y=1|x_n) + \sum_{n=1}^{KN}\log p(y=0|x'_n)\right). \quad (15.87)$$

通过不断采样真实样本和噪声样本，并用梯度下降法，可以学习参数 θ 使得 $p_\theta(x)$ 逼近于真实分布 $p_r(x)$.

噪声对比估计相当于用判别式的准则 $\mathcal{L}(\theta)$ 来训练一个生成式模型 $p_\theta(x)$，使得判别函数 D 很容易分辨出样本 x 来自哪个分布，其思想与生成对抗网络类似. 不同之处在于，在噪声对比估计中的判别函数 D 是通过贝叶斯公式计算得到，而生成对抗网络的判别函数 D 是一个需要学习的神经网络.

生成对抗网络参见第13.3节.

基于噪声对比估计的序列模型 在计算序列模型的条件概率时，我们也可以利用噪声对比估计的思想来提高计算效率 [Mnih et al., 2013, 2012]. 在序列模型中需要建模的分布是 $p_\theta(v|\tilde{h})$，原则上噪声分布 $q(v|\tilde{h})$ 应该是依赖于历史信息 $\tilde{h}$ 的条件分布，但实践中一般使用和历史信息无关的分布 $q(v)$，比如一元模型的分布.

$p_\theta(v|\tilde{h})$ 的计算参见公式(15.48).

给定历史信息 $\tilde{h}$，我们需要判断词表中每一个词 v 是来自于真实分布还是噪声分布. 令

$$p(y=1|v,\tilde{h}) = \frac{p_\theta(v|\tilde{h})}{p_\theta(v|\tilde{h}) + Kq(v)}. \quad (15.88)$$

对于一个训练序列 $\boldsymbol{x}_{1:T}$，将 $\{(\tilde{h}_t, x_t)\}_{t=1}^T$ 作为真实样本. 对于每一个 x_t，从噪声分布中抽取 K 个噪声样本 $\{x'_{t,1}, \cdots, x'_{t,K}\}$. 噪声对比估计的目标函数是

$\tilde{h}_t = \boldsymbol{x}_{1:(t-1)}$.

$$\mathcal{L}(\theta) = -\sum_{t=1}^{T}\left(\log p(y=1|x_t,\tilde{h}_t) + \sum_{k=1}^{K}\log\left(1-p(y=1|x'_{t,k},\tilde{h}_t)\right)\right). \quad (15.89)$$

为简单起见，这里省略了系数 $\frac{1}{T(K+1)}$.

虽然通过噪声对比估计，将一个 $|\mathcal{V}|$ 类的分类问题转换为一个二分类问题，但是依然需要计算 $p_\theta(v|\tilde{h})$，其中仍然涉及配分函数的计算. 为了避免计算配分函数，我们将负对数配分函数 $-\log Z(\tilde{h};\theta)$ 作为一个可学习的参数 $z_{\tilde{h}}$（即每一个 $\tilde{h}$ 对应一个参数），这样条件概率 $p_\theta(v|\tilde{h})$ 重新定义为

$$p_\theta(v|\tilde{h}) = \exp\left(s(v,\tilde{h};\theta)\right)\exp(z_{\tilde{h}}). \quad (15.90)$$

噪声对比估计方法的一个特点是会促使未归一化分布 $\exp\left(s(v,\tilde{h};\theta)\right)$ 自己学习到一个近似归一化的分布，并接近真实的数据分布 $p_r(v|\tilde{h})$ [Gutmann et al., 2010]. 也就是说，学习出来的 $\exp(z_{\tilde{h}}) \approx 1$. 这样可以直接令 $\exp(z_{\tilde{h}}) = 1, \forall\tilde{h}$，并

用未归一化的分布 $\exp\big(s(\upsilon,\tilde{h};\theta)\big)$ 来代替 $p_\theta(\upsilon|\tilde{h})$. 直接令 $\exp(z_{\tilde{h}})=1$ 不会影响模型的性能. 因为神经网络有大量的参数, 这些参数足以让模型学习到一个近似归一化的分布 [Mnih et al., 2012].

公式(15.88)可以写为

$$p(y=1|\upsilon,\tilde{h}) = \frac{\exp\big(s(\upsilon,\tilde{h};\theta)\big)}{\exp\big(s(\upsilon,\tilde{h};\theta)\big)+Kq(\upsilon)} \tag{15.91}$$

$$= \frac{1}{1+\frac{Kq(\upsilon)}{\exp\big(s(\upsilon,\tilde{h};\theta)\big)}} \tag{15.92}$$

$$= \frac{1}{1+\exp\Big(-s(\upsilon,\tilde{h};\theta)+\log\big(Kq(\upsilon)\big)\Big)} \tag{15.93}$$

$$= \frac{1}{1+\exp(-(\Delta s(\upsilon,\tilde{h};\theta)))} \tag{15.94}$$

$$= \sigma(\Delta s(\upsilon,\tilde{h};\theta)), \tag{15.95}$$

其中 σ 为 Logistic 函数, $\Delta s(\upsilon,\tilde{h};\theta)=s(\upsilon,\tilde{h};\theta)-\log(Kq(\upsilon))$ 为模型打分 (未归一化分布) 与放大的噪声分布之间的差.

在噪声对比估计中, 噪声分布 $q(\upsilon)$ 的选取也十分关键. 首先是从 $q(\upsilon)$ 中采样要十分容易. 另外, $q(\upsilon)$ 要和真实数据分布 $p_r(\upsilon|\tilde{h})$ 比较接近, 否则分类问题就变得十分容易, 不需要学习到一个接近真实分布的 $p_\theta(\upsilon|\tilde{h})$ 就可以分出数据来源了. 对自然语言的序列模型, $q(\upsilon)$ 取一元模型的分布是一个很好的选择. 每次迭代噪声样本的个数 K 取值在 $25\sim100$ 左右.

总结 基于采样的方法并不改变模型的结构, 只是近似计算参数梯度. 在训练时可以显著提高模型的训练速度, 但是在测试阶段依然需要计算配分函数. 而基于层次化 Softmax 的方法改变了模型的结构, 在训练和测试时都可以加快计算速度.

15.6 序列到序列模型

在序列生成任务中, 有一类任务是序列到序列生成任务, 即输入一个序列, 生成另一个序列, 比如机器翻译、语音识别、文本摘要、对话系统、图像标题生成等.

序列到序列 (Sequence-to-Sequence, Seq2Seq) 是一种条件的序列生成问题, 给定一个序列 $\boldsymbol{x}_{1:S}$, 生成另一个序列 $\boldsymbol{y}_{1:T}$. 输入序列的长度 S 和输出序列的长度 T 可以不同. 比如在机器翻译中, 输入为源语言, 输出为目标语言. 图15.5给

出了基于循环神经网络的序列到序列机器翻译示例，其中⟨*EOS*⟩表示输入序列的结束，虚线表示用上一步的输出作为下一步的输入.

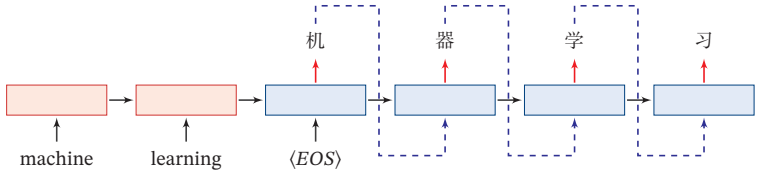


图 15.5 基于循环神经网络的序列到序列机器翻译

序列到序列模型的目标是估计条件概率

$$p_\theta(\boldsymbol{y}_{1:T}|\boldsymbol{x}_{1:S}) = \prod_{t=1}^{T} p_\theta(y_t|\boldsymbol{y}_{1:(t-1)}, \boldsymbol{x}_{1:S}), \tag{15.96}$$

其中 $\boldsymbol{y}_t \in \mathcal{V}$ 为词表 $\mathcal{V}$ 中的某个词.

给定一组训练数据 $\{(\boldsymbol{x}_{S_n}, \boldsymbol{y}_{T_n})\}_{n=1}^{N}$，我们可以使用最大似然估计来训练模型参数

$$\hat{\theta} = \arg\max_{\theta} \sum_{n=1}^{N} \log p_\theta(\boldsymbol{y}_{1:T_n}|\boldsymbol{x}_{1:S_n}). \tag{15.97}$$

一旦训练完成，模型就可以根据一个输入序列 $\boldsymbol{x}$ 来生成最可能的目标序列

$$\hat{\boldsymbol{y}} = \arg\max_{\boldsymbol{y}} p_{\hat{\theta}}(\boldsymbol{y}|\boldsymbol{x}), \tag{15.98}$$

具体的生成过程可以通过贪婪方法或束搜索来完成.

和一般的序列生成模型类似，条件概率 $p_\theta(y_t|\boldsymbol{y}_{1:(t-1)}, \boldsymbol{x}_{1:S})$ 可以使用各种不同的神经网络来实现. 这里我们介绍三种主要的序列到序列模型：基于循环神经网络的序列到序列模型、基于注意力的序列到序列模型、基于自注意力的序列到序列模型.

15.6.1 基于循环神经网络的序列到序列模型

实现序列到序列的最直接方法是使用两个循环神经网络来分别进行编码和解码，也称为编码器-解码器（Encoder-Decoder）模型.

编码器 首先使用一个循环神经网络 f_{enc} 来编码输入序列 $\boldsymbol{x}_{1:S}$ 得到一个固定维数的向量 $\boldsymbol{u}$，$\boldsymbol{u}$ 一般为编码循环神经网络最后时刻的隐状态.

$$\boldsymbol{h}_t^{\text{enc}} = f_{\text{enc}}(\boldsymbol{h}_{t-1}^{\text{enc}}, \boldsymbol{e}_{x_{t-1}}, \theta_{\text{enc}}), \qquad \forall t \in [1:S], \tag{15.99}$$

$$\boldsymbol{u} = \boldsymbol{h}_S^{\text{enc}}, \tag{15.100}$$

其中 $f_{\text{enc}}(\cdot)$ 为编码循环神经网络，可以为 LSTM 或 GRU，其参数为 θ_{enc}，$\boldsymbol{e}_x$ 为词 x 的词向量.

解码器 在生成目标序列时，使用另外一个循环神经网络 f_{dec} 来进行解码. 在解码过程的第 t 步时，已生成前缀序列为 $\boldsymbol{y}_{1:(t-1)}$. 令 $\boldsymbol{h}_t^{\text{dec}}$ 表示在网络 f_{dec} 的隐状态，$\boldsymbol{o}_t \in (0,1)^{|\mathcal{V}|}$ 为词表中所有词的后验概率，则

$$\boldsymbol{h}_0^{\text{dec}} = \boldsymbol{u}, \tag{15.101}$$

$$\boldsymbol{h}_t^{\text{dec}} = f_{\text{dec}}(\boldsymbol{h}_{t-1}^{\text{dec}}, \boldsymbol{e}_{y_{t-1}}, \theta_{\text{dec}}), \tag{15.102}$$

$$\boldsymbol{o}_t = g(\boldsymbol{h}_t^{\text{dec}}, \theta_o), \tag{15.103}$$

其中 $f_{\text{dec}}(\cdot)$ 为解码循环神经网络，$g(\cdot)$ 为最后一层为 Softmax 函数的前馈神经网络，θ_{dec} 和 θ_o 为网络参数，$\boldsymbol{e}_y$ 为 y 的词向量，y_0 为一个特殊符号，比如 $\langle EOS \rangle$.

基于循环神经网络的序列到序列模型的缺点是：1）编码向量 $\boldsymbol{u}$ 的容量问题，输入序列的信息很难全部保存在一个固定维度的向量中；2）当序列很长时，由于循环神经网络的长程依赖问题，容易丢失输入序列的信息.

长程依赖问题参见第6.5节.

15.6.2 基于注意力的序列到序列模型

为了获取更丰富的输入序列信息，我们可以在每一步中通过注意力机制来从输入序列中选取有用的信息.

注意力机制参见第8.2节.

在解码过程的第 t 步时，先用上一步的隐状态 $\boldsymbol{h}_{t-1}^{\text{dec}}$ 作为查询向量，利用注意力机制从所有输入序列的隐状态 $\boldsymbol{H}^{\text{enc}} = [\boldsymbol{h}_1^{\text{enc}}, \cdots, \boldsymbol{h}_S^{\text{enc}}]$ 中选择相关信息

$$\boldsymbol{c}_t = \text{att}(\boldsymbol{H}^{\text{enc}}, \boldsymbol{h}_{t-1}^{\text{dec}}) = \sum_{i=1}^{S} \alpha_i \boldsymbol{h}_i^{\text{enc}} \tag{15.104}$$

$$= \sum_{i=1}^{S} \text{softmax}\Big(s(\boldsymbol{h}_i^{\text{enc}}, \boldsymbol{h}_{t-1}^{\text{dec}})\Big)\boldsymbol{h}_i^{\text{enc}} \tag{15.105}$$

基于注意力的序列到序列生成过程见 https://nndl.github.io/v/sgm-seq2seq



其中 $s(\cdot)$ 为注意力打分函数.

注意力打分函数可以有多种选择，参见第8.2节.

然后，将从输入序列中选择的信息 $\boldsymbol{c}_t$ 也作为解码器 $f_{\text{dec}}(\cdot)$ 在第 t 步时的输入，得到第 t 步的隐状态

$$\boldsymbol{h}_t^{\text{dec}} = f_{\text{dec}}(\boldsymbol{h}_{t-1}^{\text{dec}}, [\boldsymbol{e}_{y_{t-1}}; \boldsymbol{c}_t], \theta_{\text{dec}}). \tag{15.106}$$

最后，将 $\boldsymbol{h}_t^{\text{dec}}$ 输入到分类器 $g(\cdot)$ 中来预测词表中每个词出现的概率.

15.6.3 基于自注意力的序列到序列模型

除长程依赖问题外，基于循环神经网络的序列到序列模型的另一个缺点是无法并行计算. 为了提高并行计算效率以及捕捉长距离的依赖关系，我们可以使用自注意力模型（Self-Attention Model）来建立一个全连接的网络结构. 本节介绍一个目前非常成功的基于自注意力的序列到序列模型：Transformer [Vaswani et al., 2017].

自注意力模型参见第8.3节.

15.6.3.1 自注意力

对于一个向量序列 $\boldsymbol{H} = [\boldsymbol{h}_1, \cdots, \boldsymbol{h}_T] \in \mathbb{R}^{D_h \times T}$，首先用自注意力模型来对其进行编码，即

$$\text{self-att}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \boldsymbol{V} \, \text{softmax}\Big(\frac{\boldsymbol{K}^{\top}\boldsymbol{Q}}{\sqrt{D_k}}\Big), \tag{15.107}$$

$$\boldsymbol{Q} = \boldsymbol{W}_q\boldsymbol{H}, \boldsymbol{K} = \boldsymbol{W}_k\boldsymbol{H}, \boldsymbol{V} = \boldsymbol{W}_v\boldsymbol{H}, \tag{15.108}$$

其中 D_k 是输入矩阵 $\boldsymbol{Q}$ 和 $\boldsymbol{K}$ 中列向量的维度，$\boldsymbol{W}_q \in \mathbb{R}^{D_k \times D_h}$，$\boldsymbol{W}_k \in \mathbb{R}^{D_k \times D_h}$，$\boldsymbol{W}_v \in \mathbb{R}^{D_v \times D_h}$ 为三个投影矩阵.

15.6.3.2 多头自注意力

自注意力模型可以看作在一个线性投影空间中建立 $\boldsymbol{H}$ 中不同向量之间的交互关系. 为了提取更多的交互信息，我们可以使用多头自注意力（Multi-Head Self-Attention），在多个不同的投影空间中捕捉不同的交互信息. 假设在 M 个投影空间中分别应用自注意力模型，有

$$\text{MultiHead}(\boldsymbol{H}) = \boldsymbol{W}_o[\text{head}_1; \cdots; \text{head}_M], \tag{15.109}$$

$$\text{head}_m = \text{self-att}(\boldsymbol{Q}_m, \boldsymbol{K}_m, \boldsymbol{V}_m), \tag{15.110}$$

$$\forall m \in \{1, \cdots, M\}, \quad \boldsymbol{Q}_m = \boldsymbol{W}_q^m\boldsymbol{H}, \boldsymbol{K} = \boldsymbol{W}_k^m\boldsymbol{H}, \boldsymbol{V} = \boldsymbol{W}_v^m\boldsymbol{H}, \tag{15.111}$$

其中 $\boldsymbol{W}_o \in \mathbb{R}^{D_h \times Md_v}$ 为输出投影矩阵，$\boldsymbol{W}_q^m \in \mathbb{R}^{D_k \times D_h}$，$\boldsymbol{W}_k^m \in \mathbb{R}^{D_k \times D_h}$，$\boldsymbol{W}_v^m \in \mathbb{R}^{D_v \times D_h}$ 为投影矩阵，$m \in \{1, \cdots, M\}$.

15.6.3.3 基于自注意力模型的序列编码

对于一个序列 $\boldsymbol{x}_{1:T}$，我们可以构建一个含有多层多头自注意力模块的模型来对其进行编码. 由于自注意力模型忽略了序列 $\boldsymbol{x}_{1:T}$ 中每个 $\boldsymbol{x}_t$ 的位置信息，因此需要在初始的输入序列中加入位置编码（Positional Encoding）来进行修正. 对于一个输入序列 $\boldsymbol{x}_{1:T} \in \mathbb{R}^{D \times T}$，令

$$\boldsymbol{H}^{(0)} = [\boldsymbol{e}_{x_1} + \boldsymbol{p}_1, \cdots, \boldsymbol{e}_{x_T} + \boldsymbol{p}_T], \tag{15.112}$$

其中 $\boldsymbol{e}_{x_t} \in \mathbb{R}^D$ 为词 x_t 的嵌入向量表示，$\boldsymbol{p}_t \in \mathbb{R}^D$ 为位置 t 的向量表示，即位置编码. $\boldsymbol{p}_t$ 可以作为可学习的参数，也可以通过下面方式进行预定义：

$$\boldsymbol{p}_{t,2i} = \sin(t/10000^{2i/D}), \tag{15.113}$$

$$\boldsymbol{p}_{t,2i+1} = \cos(t/10000^{2i/D}), \tag{15.114}$$

参见习题15-7.

其中 $\boldsymbol{p}_{t,2i}$ 表示第 t 个位置的编码向量的第 $2i$ 维，D 是编码向量的维度.

给定第 $l-1$ 层的隐状态 $\boldsymbol{H}^{(l-1)}$，第 l 层的隐状态 $\boldsymbol{H}^{(l)}$ 可以通过一个多头自注意力模块和一个非线性的前馈网络得到. 每次计算都需要残差连接以及层归一化操作. 具体计算为

$$\boldsymbol{Z}^{(l)} = \text{norm}\Big(\boldsymbol{H}^{(l-1)} + \text{MultiHead}\big(\boldsymbol{H}^{(l-1)}\big)\Big), \tag{15.115}$$

$$\boldsymbol{H}^{(l)} = \text{norm}\Big(\boldsymbol{Z}^{(l)} + \text{FFN}(\boldsymbol{Z}^{(l)})\Big), \tag{15.116}$$

其中 $\text{norm}(\cdot)$ 表示层归一化，$\text{FFN}(\cdot)$ 表示逐位置的前馈神经网络（Position-wise Feed-Forward Network），是一个简单的两层网络. 对于输入序列中每个位置上向量 $\boldsymbol{z} \in \boldsymbol{Z}^{(l)}$，

层归一化参见第7.5.2节.

$$\text{FFN}(\boldsymbol{z}) = \boldsymbol{W}_2 \text{ReLu}(\boldsymbol{W}_1 \boldsymbol{z} + \boldsymbol{b}_1) + \boldsymbol{b}_2, \tag{15.117}$$

其中 $\boldsymbol{W}_1, \boldsymbol{W}_2, \boldsymbol{b}_1, \boldsymbol{b}_2$ 为网络参数.

基于自注意力模型的序列编码可以看作一个全连接的前馈神经网络，第 l 层的每个位置都接受第 $l-1$ 层的所有位置的输出. 不同的是，其连接权重是通过注意力机制动态计算得到.

15.6.3.4 Transformer 模型

Transformer模型 [Vaswani et al., 2017] 是一个基于多头自注意力的序列到序列模型，其整个网络结构可以分为两部分：

基于Transformer的序列到序列生成过程见 https://nndl.github.io/v/sgm-seq2seq



（1）编码器只包含多层的多头自注意力（Multi-Head Self-Attention）模块，每一层都接受前一层的输出作为输入. 编码器的输入为序列 $\boldsymbol{x}_{1:S}$，输出为一个向量序列 $\boldsymbol{H}^{\text{enc}} = [\boldsymbol{h}_1^{\text{enc}}, \cdots, \boldsymbol{h}_S^{\text{enc}}]$. 然后，用两个矩阵将 $\boldsymbol{H}^{\text{enc}}$ 映射到 $\boldsymbol{K}^{\text{enc}}$ 和 $\boldsymbol{V}^{\text{enc}}$ 作为键值对供解码器使用，即

$$\boldsymbol{K}^{\text{enc}} = \boldsymbol{W}_k' \boldsymbol{H}^{\text{enc}}, \tag{15.118}$$

$$\boldsymbol{V}^{\text{enc}} = \boldsymbol{W}_v' \boldsymbol{H}^{\text{enc}}, \tag{15.119}$$

其中 $\boldsymbol{W}_k'$ 和 $\boldsymbol{W}_v'$ 为线性映射的参数矩阵.

（2）解码器是通过自回归的方式来生成目标序列. 和编码器不同，解码器由以下三个模块构成：

y_0 为一个特殊符号.

为了提高训练效率，这里的自注意力模型实际上是掩蔽自注意力，即阻止每个位置看到它右边的词.

a）掩蔽自注意力模块：第 t 步时，先使用自注意力模型对已生成的前缀序列 $\boldsymbol{y}_{0:(t-1)}$ 进行编码得到 $\boldsymbol{H}^{\text{dec}} = [\boldsymbol{h}_1^{\text{dec}}, \cdots, \boldsymbol{h}_t^{\text{dec}}]$.

b）解码器到编码器注意力模块：将 $\boldsymbol{h}_t^{\text{dec}}$ 进行线性映射得到 $\boldsymbol{q}_t^{\text{dec}}$. 将 $\boldsymbol{q}_t^{\text{dec}}$ 作为查询向量，通过键值对注意力机制来从输入 $(\boldsymbol{K}^{\text{enc}}, \boldsymbol{V}^{\text{enc}})$ 中选取有用的信息.

c）逐位置的前馈神经网络：使用一个前馈神经网络来综合得到所有信息.

将上述三个步骤重复多次，最后通过一个全连接前馈神经网络来计算输出概率. 图15.6给出了 Transformer 的网络结构示例，其中 $N\times$ 表示重复 N 次，“Add & Norm” 表示残差连接和层归一化. 在训练时，为了提高效率，我们通常将右移的目标序列（Right-Shifted Output）$\boldsymbol{y}_{0:(T-1)}$ 作为解码器的输入，即在第 t 个位置的输入为 y_{t-1}. 在这种情况下，可以通过一个掩码（Mask）来阻止每个位置选择其后面的输入信息. 这种方式称为掩蔽自注意力（Masked Self-Attention）.

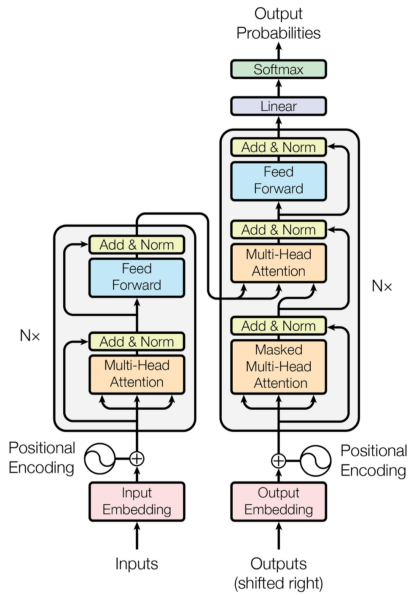


图 15.6 Transformer 网络结构（图片来源 [Vaswani et al., 2017]）

15.7 总结和深入阅读

序列生成模型主要解决序列数据的密度估计和生成问题，是一种在实际应用中十分重要的模型. 目前主流的序列生成模型都是自回归生成模型.

最早的深度序列模型是神经网络语言模型. [Bengio et al., 2003] 最早提出了基于前馈神经网络的语言模型，随后 [Mikolov et al., 2010] 利用循环神经网络来